

High-Performance Large-Scale Image Recognition Without Normalization

New SOTA validation accuracies on ImageNet by DeepMind

이미지 팀 : 김병현, 박동훈, 안종식, 이찬혁, 허다운, 홍은기
발표자 : 박동훈

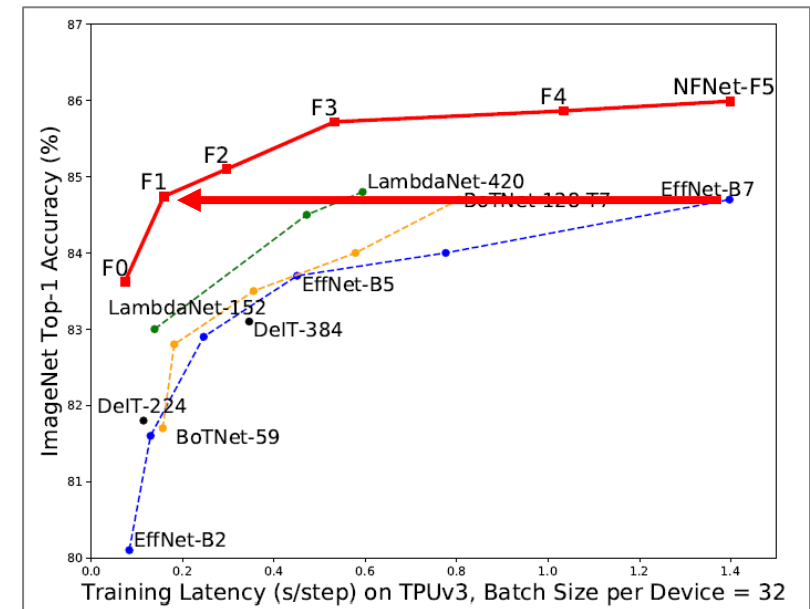
Contents

1. Performance
2. Batch Normalization: pros and cons
3. Previous Normalizer-Free Networks
4. Proposed Method : Adaptive gradient clipping
5. Experimental Results
6. Conclusion

Performance

- EfficientNet-B7 on ImageNet **8.7x** faster to train
- New state-of-the-art top-1 accuracy of **86.5%**.
 - After finetuning on ImageNet after pretraining on 300 million labeled images, It achieves **89.2%**
- Image Classification on ImageNet^[1]

1	Meta Pseudo Labels (EfficientNet-L2)	90.2%	98.8%	480M	✓	Meta Pseudo Labels			2021
2	Meta Pseudo Labels (EfficientNet-B6-Wide)	90%	98.7%	390M	✓	Meta Pseudo Labels			2021
3	NFNet-F4+	89.2%		527M	✓	High-Performance Large-Scale Image Recognition Without Normalization			2021



Batch Normalization_(previous knowledge)

Train

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

Also calculate exponential moving avg & var

$$\mu \leftarrow (1 - \alpha)\mu + \alpha\mu_{\mathcal{B}}$$

$$\sigma^2 \leftarrow (1 - \alpha)\sigma^2 + \alpha\sigma_{\mathcal{B}}^2$$

Test

$$\text{BN}_{\text{test}}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

- The change in the distributions of each layers occurs ‘**Covariant Shift**’ [2]

Batch Normalization 효과

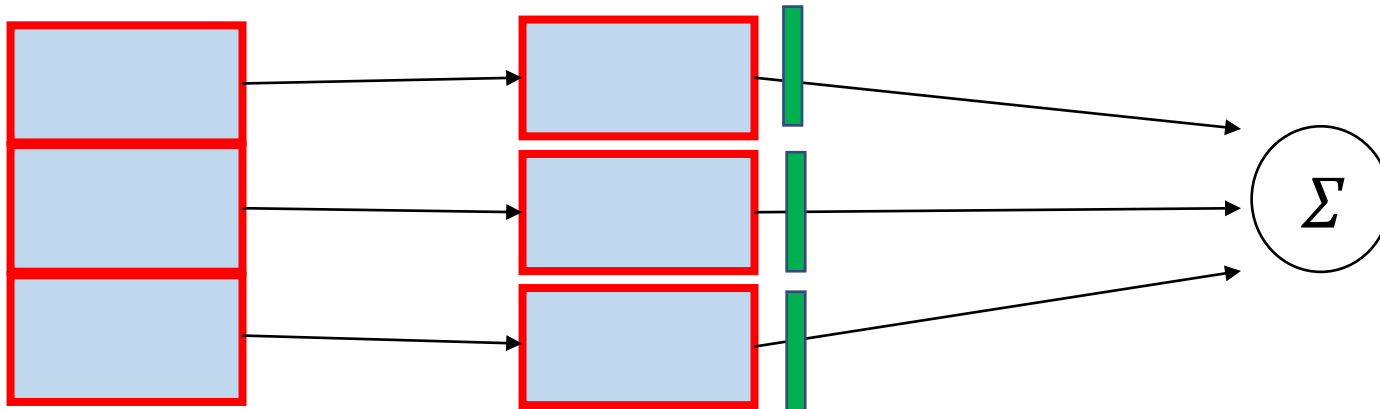
- Downscales residual branch
- Regularizing effect
- Eliminates mean-shift
- Efficient large-batch training

Batch Normalization – Bad 👎

- First, it is a surprisingly **expensive computational primitive**, which incurs memory overhead
→ 계산 과부하
- **Discrepancy** between the behaviors of the model during **training and at inference time**.
→ 학습과 추론 동작 상이
- **Breaks the independence** between training examples **in the minibatch**
→ Mini-batch의 독립성을 깨트린다.

Batch Normalization - Bad 🙅

- Can train residual networks with large learning rate, But **only benefit if batch size is also large**.
→ 배치 정규화는 큰 lr를 사용 가능하게 하지만, 배치 사이즈도 커야 효과 있다.
- Batch normalization is often the cause of subtle implementation **errors**, **especially during distributed training**(Pham et al., 2019)
→ (실무자들의 의견에 의하면) 배치 정규화를 하면 HW에 따라 결과가 매번 달랐으며, 특히 분산 학습에서 미세한 구현 에러가 발생하였다.

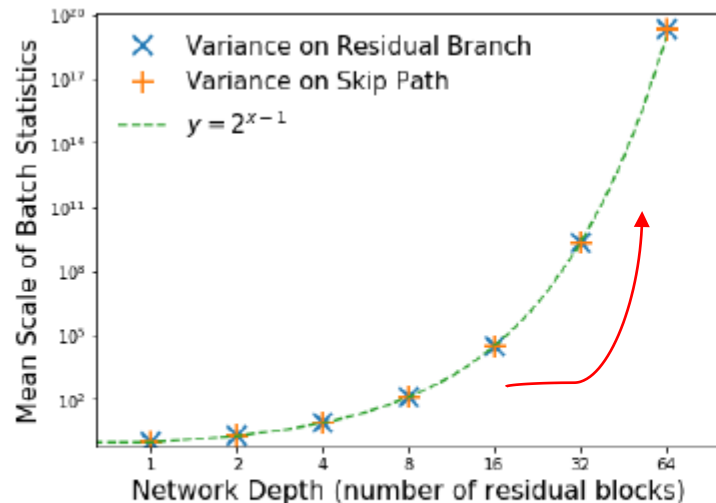


Q & A

Previous Normalizer-Free Networks

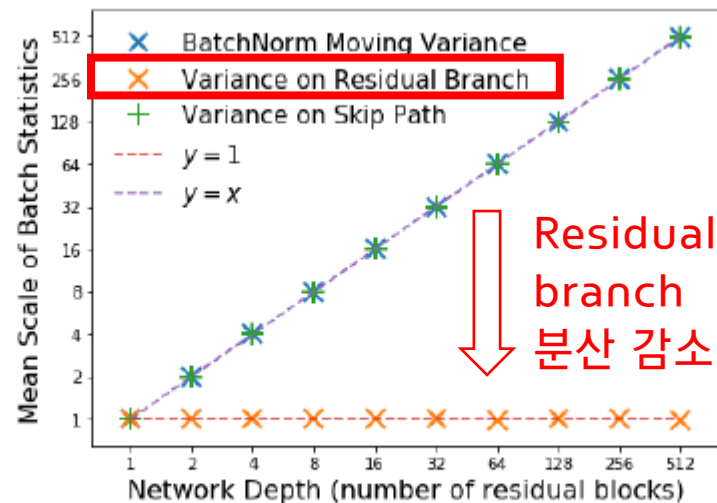
"Batch Normalization Biases Residual Blocks Towards the Identity Function in Deep Networks"
(In NIPS 2020)

Fully connected linear
unnormalized residual network



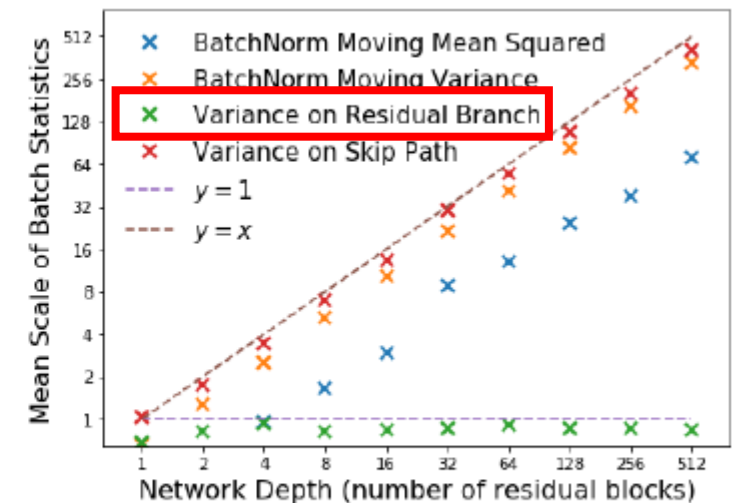
(a)

Fully connected linear
normalized residual network



(b)

Normalized convolutional
residual network

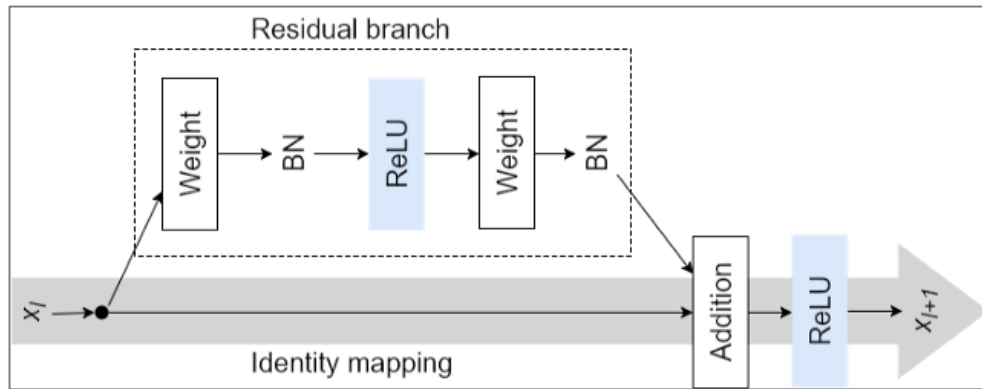


(c)

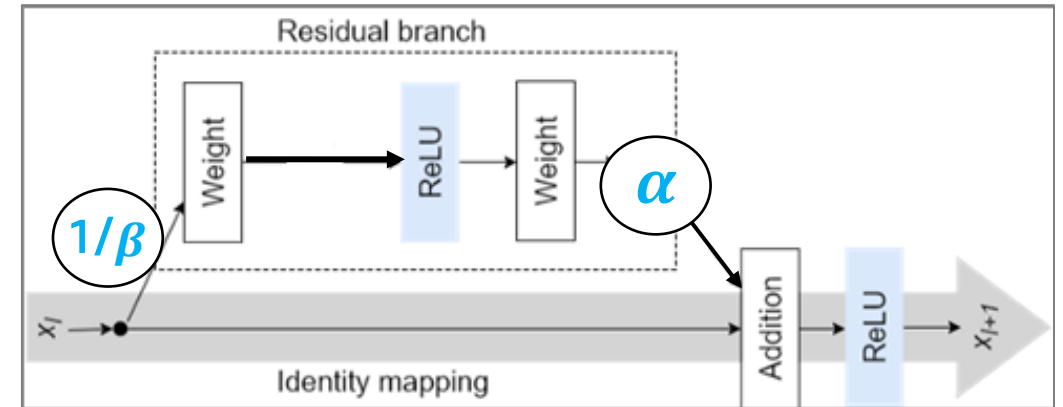
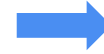
"If our theory is correct, it should be possible to train deep residual networks without normalization, simply by downscaling the residual branch."

Previous Normalizer-Free Networks

“Normalizer-Free ResNets”(NF-ResNets)(Brock et al., 2021)



Original residual unit proposed by K. He et al



- Residual block : $h_{i+1} = h_i + \alpha f_i(h_i / \beta_i)$
- h_i : residual block의 입력
- $Var(f_i(z)) = Var(z)$
- α 는 Residual Block이후 곱해주는 값. 분산을 줄여 주기 위함. e.g. $\alpha = 0.2$
- $\beta_i = \sqrt{Var(h_i)}$ where $Var(h_{i+1}) = Var(h_i) + \alpha^2$

Previous Normalizer-Free Networks

그외

- Weight Standardization Standardization (Huang et al., 2017; Qiao et al., 2019))

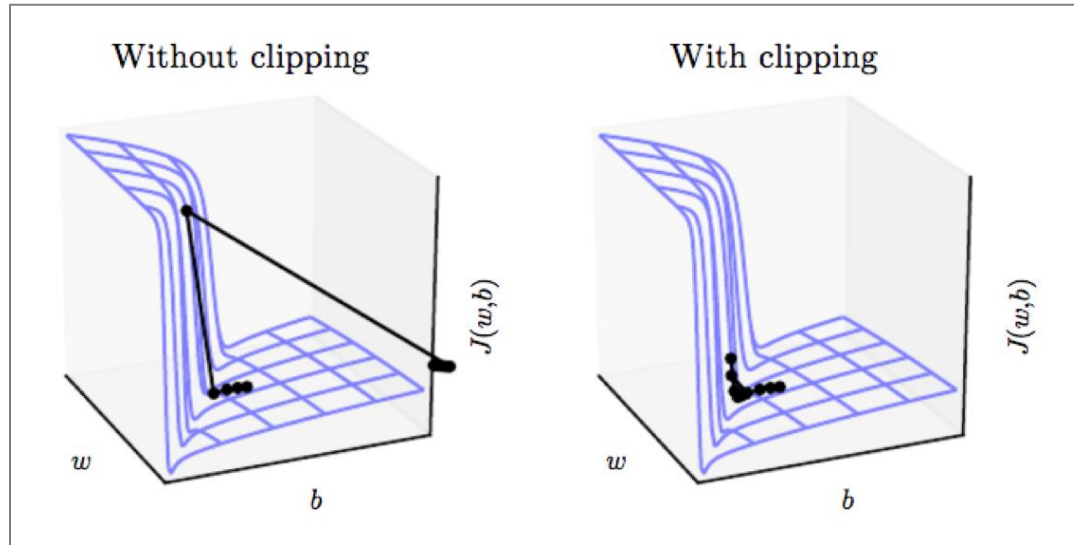
$$W_{ij} = \frac{W_{ij} - \mu_i}{\sqrt{N}\sigma_i}$$

$$\mu_i = (1/N) \sum_j W_{ij} , \sigma_i^2 = (1/N) \sum_j (W_{ij} - \mu_i)^2$$

- Dropout (Srivastava et al., 2014)
- Stochastic Depth (Huang et al., 2016))

Q & A

Gradient Clipping_(previous knowledge)



- 경사 하강(gradient descent)이 가파른 절벽에서 합리적으로 수행될 수 있도록 돕는다.
- RNN 계열의 모델 학습에 널리 쓰인다.
- Hard to tuning threshold(λ)
- But enable us to train higher batch size

$$\frac{\partial \epsilon}{\partial \theta} \leftarrow \begin{cases} \frac{\text{threshold}}{\|\hat{g}\|} \hat{g} & \text{if } \|\hat{g}\| \geq \text{threshold} \\ \hat{g} & \text{otherwise} \end{cases}$$

where $\hat{g} = \frac{\partial \epsilon}{\partial \theta}$.

기울기 norm $\|g\|$ 이 thres 보다 클 경우,
→ 정규화 해서 thres 크기 만큼으로 벡터 크기를 수정

$\hat{g}$: Gradient vector

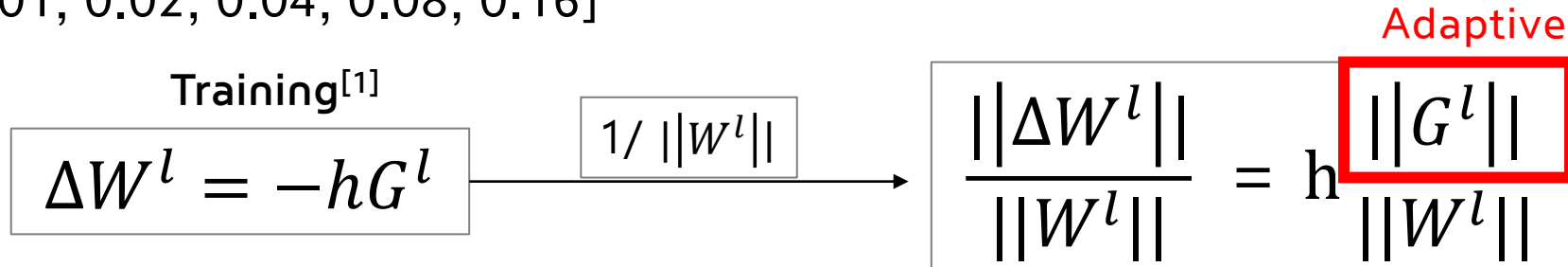
ϵ : Loss

θ : Vector

Adaptive Gradient Clipping

$$G_i^\ell \rightarrow \begin{cases} \lambda \frac{\|W_i^\ell\|_F^*}{\|G_i^\ell\|_F} G_i^\ell & \text{if } \frac{\|G_i^\ell\|_F}{\|W_i^\ell\|_F^*} > \lambda, \\ G_i^\ell & \text{otherwise.} \end{cases} \quad (3)$$

- 비율 $\frac{\|G^l\|}{\|W^l\|}$ 이 학습의 단위가 될 수 있다는 것에 영감을 받았다.
- $W^l \in R^{N \times M}$: l^{th} 번째 계층의 가중치 행렬
- $G^l \in R^{N \times M}$: W^l 에 대응하는 기울기
- $\|W^l\|_F = \sqrt{\sum_i^N \sum_j^M (W_{i,j}^l)^2}$, $\|W^l\| = \max(\|W^l\|_F, \epsilon)$, $\epsilon = 10^{-3}$
- $\lambda = [0.01, 0.02, 0.04, 0.08, 0.16]$



Experimental Result

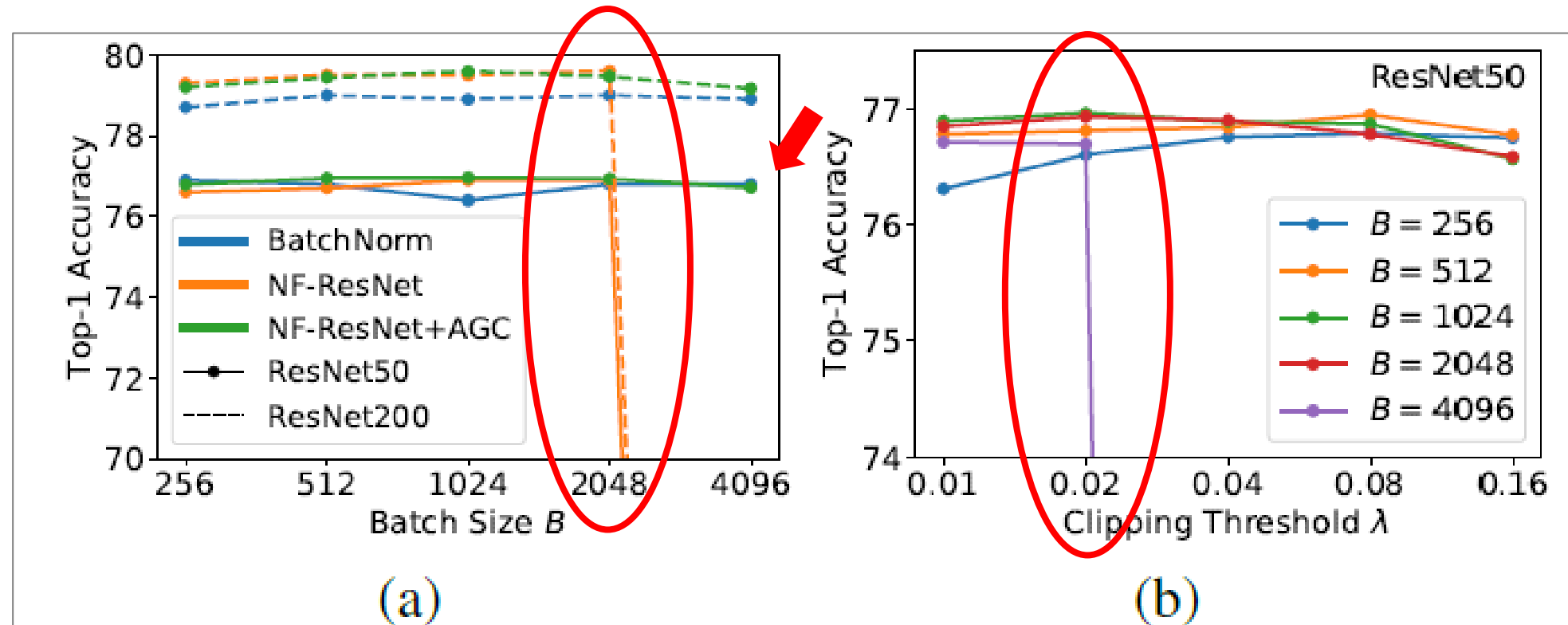


Figure 2. (a) AGC efficiently scales NF-ResNets to larger batch sizes. (b) The performance across different clipping thresholds λ .

Model Detail

Table 1. NFNet family depths, drop rates, and input resolutions.

Variant	Depth	Dropout	Train	Test
F0	[1, 2, 6, 3]	0.2	192px	256px
F1	[2, 4, 12, 6]	0.3	224px	320px
F2	[3, 6, 18, 9]	0.4	256px	352px
F3	[4, 8, 24, 12]	0.4	320px	416px
F4	[5, 10, 30, 15]	0.5	384px	512px
F5	[6, 12, 36, 18]	0.5	416px	544px
F6	[7, 14, 42, 21]	0.5	448px	576px

[Training Detail]

- Softmax cross-entropy loss with label smoothing of 0.1
- Stochastic gradient descent with Nesterov's momentum 0.9
- Weight decay coefficient 2×10^{-5}
- Dropout, Stochastic Depth(0.25)

Table 3. ImageNet Accuracy comparison for NFNets and a representative set of models, including SENet (Hu et al., 2018), LambdaNet, (Bello, 2021), BoTNet (Srinivas et al., 2021), and DeIT (Touvron et al., 2020). Except for results using SAM, our results are averaged over three random seeds. Latencies are given as the time in milliseconds required to perform a single full training step on TPU or GPU (V100).

Model	#FLOPs	#Params	Top-1	Top-5	TPUv3 Train	GPU Train
ResNet-50	4.10B	26.0M	78.6	94.3	41.6ms	35.3ms
EffNet-B0	0.39B	5.3M	77.1	93.3	51.1ms	44.8ms
SENet-50	4.09B	28.0M	79.4	94.6	64.3ms	59.4ms
NFNet-F0	12.38B	71.5M	83.6	96.8	73.3ms	56.7ms
EffNet-B3	1.80B	12.0M	81.6	95.7	129.5ms	116.6ms
LambdaNet-152	—	51.5M	83.0	96.3	138.3ms	135.2ms
SENet-152	19.04B	66.6M	83.1	96.4	149.9ms	151.2ms
BoTNet-110	10.90B	54.7M	82.8	96.3	181.3ms	—
NFNet-F1	35.54B	132.6M	84.7	97.1	158.5ms	133.9ms
EffNet-B4	4.20B	19.0M	82.9	96.4	245.9ms	221.6ms
BoTNet-128-T5	19.30B	75.1M	83.5	96.5	355.2ms	—
NFNet-F2	62.59B	193.8M	85.1	97.3	295.8ms	226.3ms
SENet-350	52.90B	115.2M	83.8	96.6	593.6ms	—
EffNet-B5	9.90B	30.0M	83.7	96.7	450.5ms	458.9ms
LambdaNet-350	—	105.8M	84.5	97.0	471.4ms	—
BoTNet-77-T6	23.30B	53.9M	84.0	96.7	578.1ms	—
NFNet-F3	114.76B	254.9M	85.7	97.5	532.2ms	524.5ms
LambdaNet-420	—	124.8M	84.8	97.0	593.9ms	—
EffNet-B6	19.00B	43.0M	84.0	96.8	775.7ms	868.2ms
BoTNet-128-T7	45.80B	75.1M	84.7	97.0	804.5ms	—
NFNet-F4	215.24B	316.1M	85.9	97.6	1033.3ms	1190.6ms
EffNet-B7	37.00B	66.0M	84.7	97.0	1397.0ms	1753.3ms
DeIT 1000 epochs	—	87.0M	85.2	—	—	—
EffNet-B8+MaxUp	62.50B	87.4M	85.8	—	—	—
NFNet-F5	289.76B	377.2M	86.0	97.6	1398.5ms	2177.1ms
NFNet-F5+SAM	289.76B	377.2M	86.3	97.9	1958.0ms	—
NFNet-F6+SAM	377.28B	438.4M	86.5	97.9	2774.1ms	—

Conclusion

- 배치 정규화를 적용하지 않고도, 큰 배치 사이즈로 학습 할 때 배치 정규화를 적용한 모델의 성능을 뛰어 넘는 최초의 모델
- 배치 정규화 적용한 모델과 성능은 비슷하면서도 빠르게 학습할 수 있다.
- AGC 기법을 적용한 family models을 만들었다
- 정규화 없는 모델이 (이미지 넷과 같은 모델을 학습 한 후) Finetuning 할 때 되려 더 좋은 성능을 나타낸 다는 것을 보였다.

개인적인 생각

- 해당 팀은 정규화 없는 학습 방식 관련해서 연구를 많이 한 팀.
- 학습 속도를 빠르게 해서 모델 검증을 하고 싶음.
- 정규화 없이 학습을 하려 했으나, 배치 사이즈를 크게 하니 성능이 되려 안 좋게 나옴.
- 여러 아이디어로 실험을 해보고 나온 결과.

감사합니다